МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Реализация численных алгоритмов

Студент
гр. БИН-26-2 _____ Е.С. Владимиренко

Преподаватель _____ М.В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Наибольший общий делитель

Пользователь вводит два целых числа. Если хотя бы одно отрицательное или оба равны нулю, выводится ошибка. Иначе находится наибольший общий делитель алгоритмом Евклида.

Алгоритм решения задания представлен на рисунке 1.

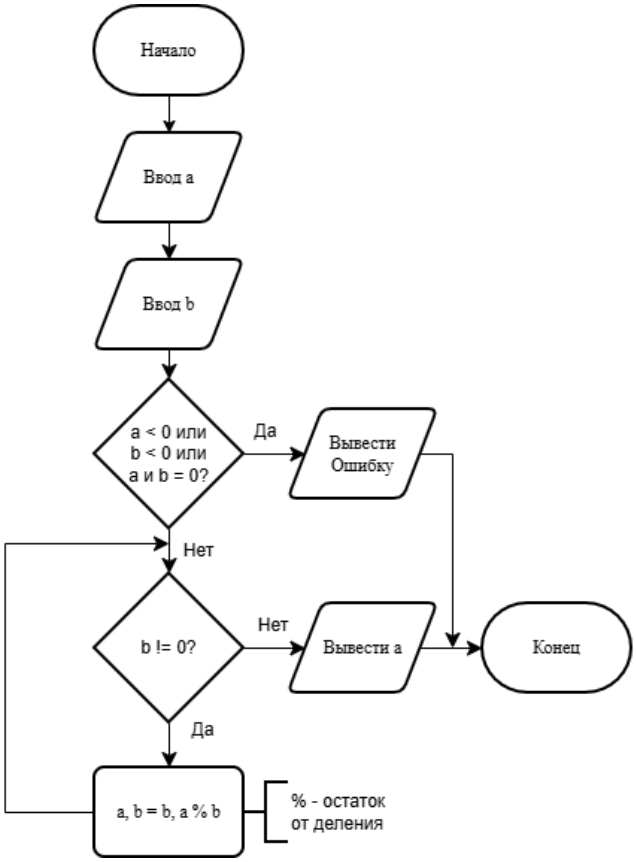


Рисунок 1 – Блок-схема программы для задания 1

На блок-схеме показана проверка входных данных и цикл алгоритма Евклида.

Текст программы и пример её работы представлены на рисунке 2.

task1.py > ...

```
1  a = int(input())
2  b = int(input())
3  if a < 0 or b < 0 or (a == 0 and b == 0):
4      print('Ошибка')
5  else:
6      while b != 0:
7          a, b = b, a % b
8      print(f'НОД: {a}')
```

Проблемы | Выходные данные | Консоль отладки | Терминал | Порты

```
PS C:\Users\sonni\OneDrive\Рабочий стол\lab6> & C:\Users\sonni\
84
30
НОД: 6
```

Рисунок 2 – Текст программы и результат работы для задания 1

Программа находит НОД двух чисел с помощью алгоритма Евклида.

1.2 Проверка числа на простоту

Пользователь вводит целое число. Необходимо определить, является ли оно простым.

Алгоритм решения задания представлен на рисунке 3.

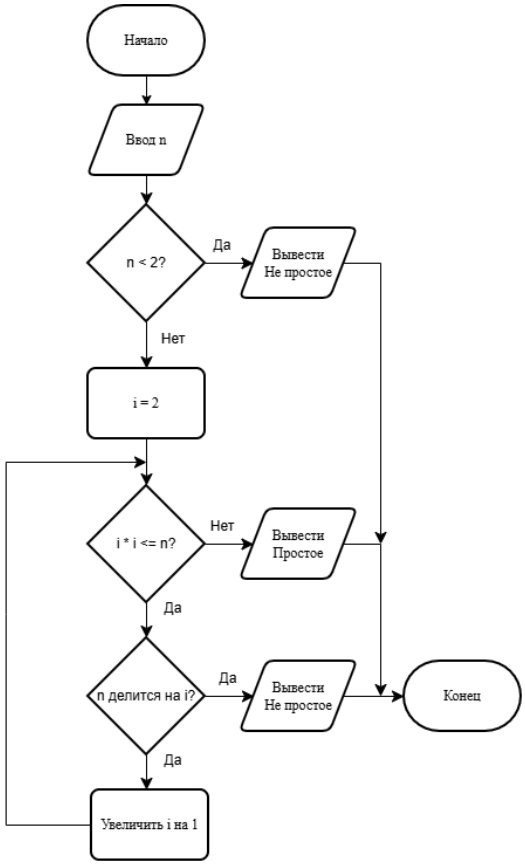


Рисунок 3 – Блок-схема программы для задания 2

На блок-схеме показана проверка числа на простоту путём перебора делителей.

Текст программы и пример её работы представлены на рисунке 4.

```
task2.py > ...
1   n = int(input())
2   if n < 2:
3       print('Не простое')
4   else:
5       is_prime = True
6       i = 2
7       while i**2<= n:
8           if n % i == 0:
9               is_prime = False
10              break
11          i += 1
12      if is_prime:
13          print('Простое')
14      else:
15          print('Не простое')
```

```
Проблемы  Выходные данные  Консоль отладки  Терминал  Порты

PS C:\Users\sonni\OneDrive\Рабочий стол\lab6> & C:\Users\sonni\
42
Не простое
```

Рисунок 4 – Текст программы и результат работы для задания 2

Программа определяет, является ли введённое число простым.

1.3 Разложение на простые множители

Пользователь вводит целое число. Если число меньше 2, выводится ошибка. Иначе число разлагается на простые множители.

Алгоритм решения задания представлен на рисунке 5.

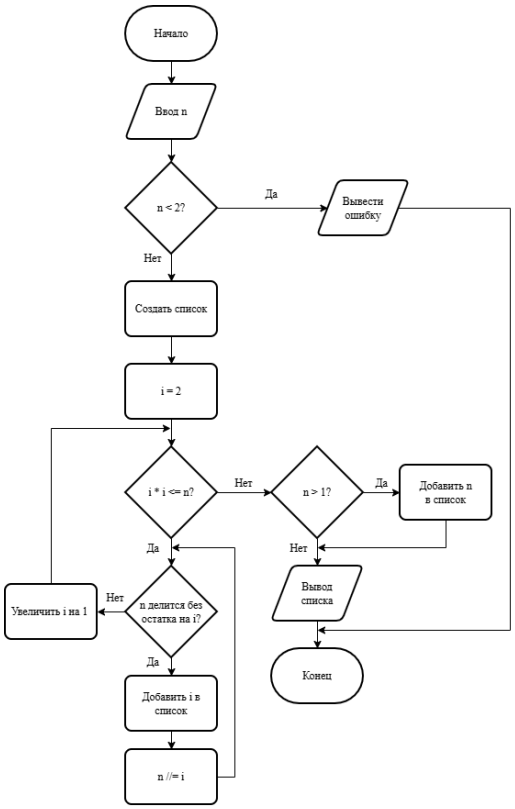


Рисунок 5 – Блок-схема программы для задания 3

На блок-схеме показан процесс последовательного деления числа на простые множители.

Текст программы и пример её работы представлены на рисунке 6.

task3.py > ...

```
1  n = int(input())
2  if n < 2:
3      print('Ошибка')
4  else:
5      factors = []
6      i = 2
7      while i**2 <= n:
8          while n % i == 0:
9              factors.append(i)
10             n //= i
11         i += 1
12     if n > 1:
13         factors.append(n)
14     print(*factors)
```

Проблемы | Выходные данные | Консоль отладки | Терминал | Порты

```
PS C:\Users\sonni\OneDrive\Рабочий стол\lab6> & C:\Users\sonni
60
2 2 3 5
```

Рисунок 6 – Текст программы и результат работы для задания 3

Программа разлагает число на простые множители и выводит их.

1.4 Цифры числа

Пользователь вводит целое число. Если число отрицательное, выводится ошибка. Иначе определяется количество цифр и их сумма.

Алгоритм решения задания представлен на рисунке 7.

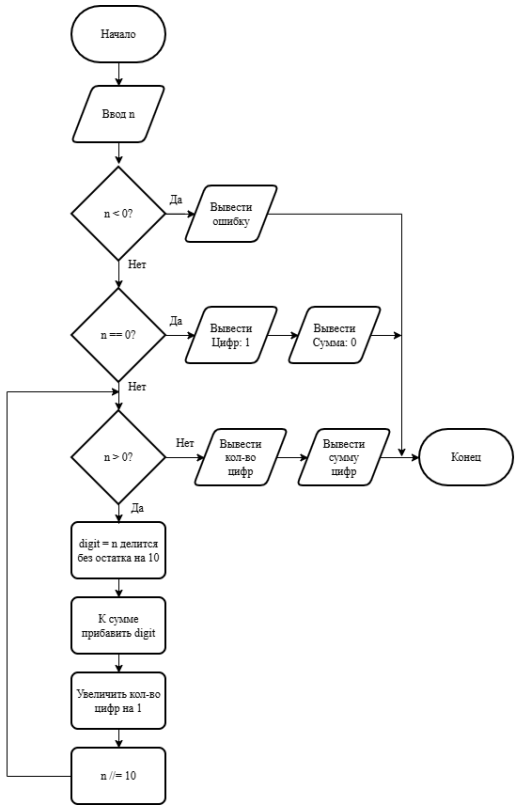


Рисунок 7 – Блок-схема программы для задания 4

На блок-схеме показан поразрядный разбор числа для подсчёта цифр и их суммы.

Текст программы и пример её работы представлены на рисунке 8.

```
task4.py > ...
1   n = int(input())
2   if n < 0:
3       print('Ошибка')
4   else:
5       if n == 0:
6           print('Цифр: 1')
7           print('Сумма цифр: 0')
8       else:
9           count = 0
10          sum_n = 0
11          while n > 0:
12              digit = n % 10
13              sum_n += digit
14              count += 1
15              n //= 10
16          print(f'Цифр: {count}')
17          print(f'Сумма цифр: {sum_n}')
```

```
Проблемы  Выходные данные  Консоль отладки  Терминал  Порты

PS C:\Users\sonni\OneDrive\Рабочий стол\lab6> & C:\Users\sonni'
4050
Цифр: 4
Сумма цифр: 9
```

Рисунок 8 – Текст программы и результат работы для задания 4

Программа подсчитывает количество цифр числа и их сумму.

1.5 Таблица расстояний

Пользователь вводит целое число от 1 до 10. Необходимо построить таблицу расстояний между точками с координатами от 0 до n-1.

Алгоритм решения задания представлен на рисунке 9.

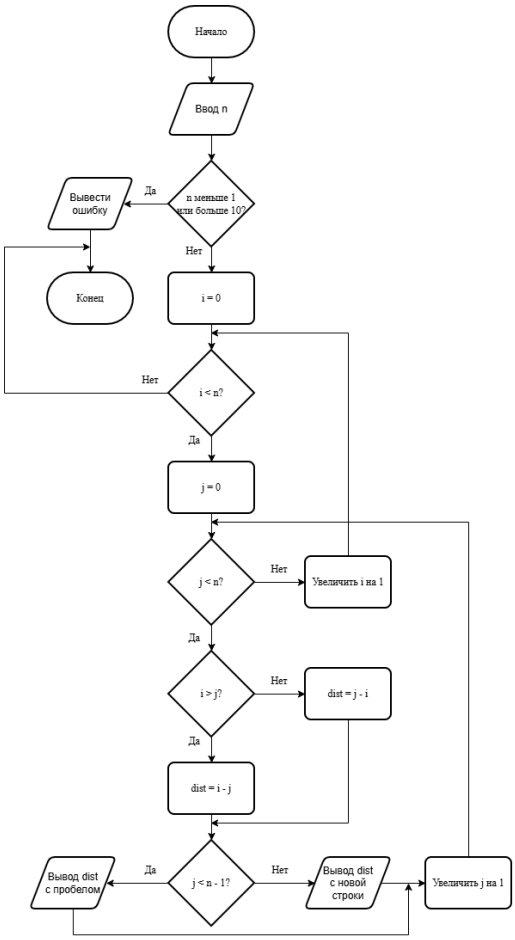


Рисунок 9 – Блок-схема программы для задания 5

На блок-схеме показаны вложенные циклы построения таблицы расстояний.

Текст программы и пример её работы представлены на рисунке 10.

```
task5.py > ...
1   n = int(input())
2   if n < 1 or n > 10:
3       print('Ошибка')
4   else:
5       for i in range(n):
6           for j in range(n):
7               if i > j:
8                   dist = i - j
9               else:
10                  dist = j - i
11              if j < n - 1:
12                  print(dist, end = ' ')
13              else:
14                  print(dist)
```

```
Проблемы  Выходные данные  Консоль отладки  Терминал  Порты

PS C:\Users\sonni\OneDrive\Рабочий стол\lab6> & C:\Users\sonni\
4
0 1 2 3
1 0 1 2
2 1 0 1
3 2 1 0
```

Рисунок 10 – Текст программы и результат работы для задания 5

Программа строит таблицу расстояний между точками на прямой.